B. Bhargavi **Reg.No:192311136**

1. (CO1) A cleaning robot works in a 3×3 room grid. It starts at cell S and must reach the dirty cell G. One cell is blocked by a chair (obstacle).

Constraints:

- **Movement allowed: Up, Down, Left, Right only**
- **The robot cannot pass through the blocked cell**
- **Each move costs 1**

Task: Apply Breadth-First Search (BFS) to find the shortest path from S to G. Show the order in which cells are expanded and state the final path and its cost.

Answer: **Breadth-First Search (BFS) – Cleaning Robot**

Problem

A cleaning robot is working in a **3 × 3 grid**.

S = Start position

G = Dirty cell (Goal)

X = Blocked cell (Chair/Obstacle)

Example Grid:

S A B

C X D

E F G

The robot can move only:

- Up
- Down
- Left
- Right

Each move costs **1**.

Step 1: BFS Expansion Order

BFS checks all nearby cells level by level.

Expansion order:

S → A → C → B → E → D → F → G

The blocked cell **X** is skipped because the robot cannot move through it.

Step 2: Shortest Path

The shortest path from **S** to **G** is:

S → A → B → D → G

Step 3: Path Cost

Moves:

S → A = 1

A → B = 1

B → D = 1

D → G = 1

Total Cost = 4

Result

Algorithm Used: Breadth-First Search (BFS)

Expanded Cells: S → A → C → B → E → D → F → G

Shortest Path: S → A → B → D → G

Total Cost: 4

Conclusion

Breadth-First Search (BFS) explores the grid level by level and always finds the **shortest path** when all moves have the same cost. The robot avoids the blocked cell and reaches the dirty cell in **4 moves**.

2. (CO1) An intelligent vacuum-cleaner agent perceives whether its current room is Clean or Dirty and can perform the actions: Suck, Move Left, or Move Right.

Task: (a) Describe this as an intelligent agent by identifying its Percepts, Actions, Environment, and Sensors/Actuators (PEAS). (b) State whether the environment is fully or partially observable, and deterministic or stochastic, with reasons.

Answer. Intelligent Vacuum Cleaner Agent

(a) PEAS Description

P – Performance Measure

- Clean all rooms
- Finish cleaning quickly
- Use less power
- Avoid unnecessary movements

E – Environment

- Two rooms (Room A and Room B)
- Rooms may be **Clean** or **Dirty**

A – Actuators (Actions)

- **Suck** – Clean the dirty room
- **Move Left**
- **Move Right**

S – Sensors (Percepts)

- Detect the current room
- Check whether the room is **Clean** or **Dirty**

(b) Environment Type

1. Observability: Fully Observable

Reason:

The vacuum cleaner can detect whether its current room is clean or dirty, so it has all the information it needs to decide its next action.

2. Deterministic or Stochastic: Deterministic

Reason:

Every action gives a predictable result.

- **Suck** cleans the room.
- **Move Left** moves to the left room.
- **Move Right** moves to the right room.

There is no random or unexpected outcome.

Conclusion

The intelligent vacuum cleaner uses its **sensors** to detect the room's condition and its **actuators** to clean or move. The environment is **fully observable** and **deterministic**, so the agent can easily choose the correct action.

3. (CO1) A robot is placed in a small maze represented as a 3×4 grid. It must find a path from the Start cell (S) to the Goal cell (G). Some cells are blocked by walls.

Constraints:

- **Movement allowed: Up, Down, Left, Right only**
- **The robot cannot pass through blocked cells**
- **Each move costs 1**

Task: Apply Depth-First Search (DFS) to find a path from S to G. Show the order in which cells are expanded (using a stack), state the final path found, and explain why the path found by DFS may not always be the shortest.

Answer: Depth-First Search (DFS) – Simple Answer

Given Grid

S A B C

D X E F

H I J G

- S = Start
- G = Goal
- X = Blocked cell (Wall)
- Movement: Up, Down, Left, Right
- Cost of each move = 1

Step 1: DFS Expansion Order (Using Stack)

DFS explores one path as deep as possible before backtracking.

Expanded Order:

S → A → B → C → F → G

The blocked cell X is skipped.

Step 2: Final Path

S → A → B → C → F → G

Step 3: Path Cost

S → A = 1

A → B = 1

B → C = 1

C → F = 1

F → G = 1

Total Cost = 5

Step 4: Why DFS May Not Find the Shortest Path

DFS always follows one path until it reaches the goal or a dead end. It does not compare all possible paths. Therefore, it may find a longer path even if a shorter path exists.

Conclusion

Algorithm Used: Depth-First Search (DFS)

Expanded Cells: S → A → B → C → F → G

Final Path: S → A → B → C → F → G

Total Cost: 5

DFS is simple and uses a stack, but it does not always find the shortest path.